

Computational Engineering

universitat politècnica de catalunya

Assignment 2: Convection-diffusion transport equation

Antonio Luque López

MUEA

27/03/2025

Contents

1 Problem Specification **2**

2 Numerical Methodology **3**

3 Code Structure **4**

4 Code Verification **5**

5 Code Results **5**

6 Numerical studies **6**

6.1 Numerical Scheme Comparison 6

6.2 Mesh convergence study 7

7 Annexes **9**

1 Problem Specification

This study focuses on the numerical resolution of the convective-diffusive transport problem proposed by Smith-Hutton. It considers a two-dimensional, steady-state problem without any source term, where a scalar quantity ϕ (e.g., temperature or concentration) is transported within a rectangular domain.

The domain is defined as a rectangle with $x \in [-1, 1]$ and $y \in [0, 1]$. The scalar field $\phi(x, y)$ is transported by a prescribed velocity field $\vec{u} = (u(x, y), v(x, y))$, which introduces a recirculating flow pattern inside the domain. The velocity components are given by:

$$u(x, y) = 2y(1 - x^2), \quad v(x, y) = -2x(1 - y^2)$$

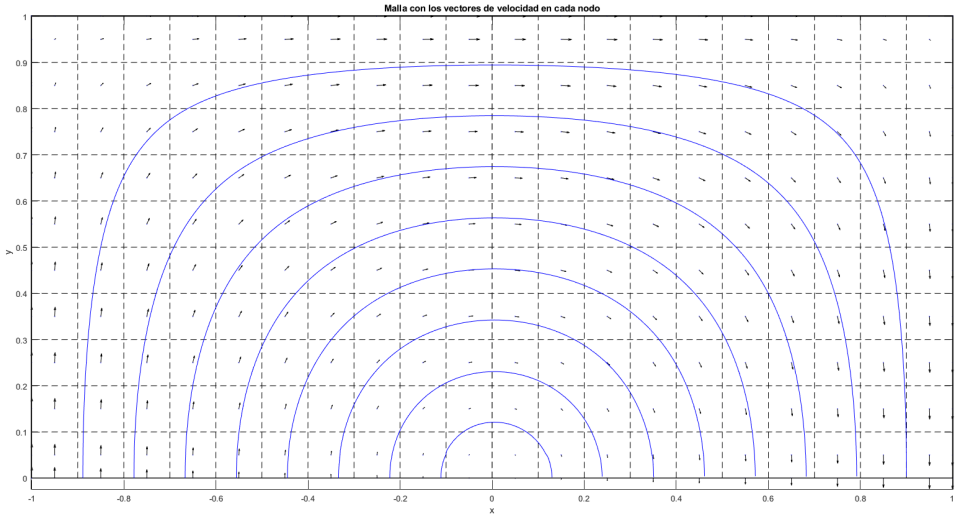


Figure 1: Domain mesh with velocity field. Streamlines plotted. N=20, M=10

The governing equation is the 2D steady-state, no-source convection-diffusion equation:

$$\nabla(\rho\vec{v}\phi) = \nabla(\Gamma\nabla\phi) \tag{1}$$

where ρ is the fluid density (assumed constant) and Γ is the diffusion coefficient. The problem is solved for different values of the ratio ρ/Γ, which characterizes the flow regime: diffusion-dominated or convection-dominated.

The boundary conditions are defined as follows:

- **Inlet**: $y = 0$, $x \in [-1, 0]$

$$\phi(x, 0) = 1 + \tanh(\alpha(2x + 1))$$

- **Outlet**: $y = 0$, $x \in (0, 1]$

$$\frac{\partial\phi}{\partial y} = 0$$

- **Elsewhere** (walls and top boundary):

$$\phi(x, y) = 1 - \tanh(\alpha)$$

where the parameter $\alpha = 10$ controls the steepness of the inlet profile.

The objective is to numerically compute the scalar field $\phi(x, y)$ for different values of the Peclet number, and to analyze the behavior of the solution under varying transport regimes.

2 Numerical Methodology

To solve the convection-diffusion equation numerically, the finite volume method is applied to discretize the computational domain. In this method, the convective and diffusive terms are evaluated at the cell faces, whereas the dependent variable ϕ is known at the cell centers. To estimate the value of ϕ at the faces, different numerical schemes are used. The accuracy of these schemes depends on the number of neighboring nodes involved.

The final discretized form of the equation becomes:

$$a_P \phi_P = a_E \phi_E + a_W \phi_W + a_N \phi_N + a_S \phi_S + b$$

Where the coefficients a_i :

$$\begin{aligned}
a_E &= D_e \cdot A(|P_e|) + \max(-F_e, 0) \\
a_W &= D_w \cdot A(|P_w|) + \max(F_w, 0) \\
a_N &= D_n \cdot A(|P_n|) + \max(-F_n, 0) \\
a_S &= D_s \cdot A(|P_s|) + \max(F_s, 0) \\
a_P &= a_E + a_W + a_N + a_S + \rho_P^n \frac{\Delta x\, \Delta y}{\Delta t} \\
b &= \rho_P^n \frac{\Delta x\, \Delta y}{\Delta t} \phi_P^n + S_P^{n+1} \Delta x\, \Delta y
\end{aligned}$$

And where $A(|P_e|)$ for each numerical scheme:

Numerical scheme	$A(\|P\|)$
UDS	1
CDS	$1 - 0.5\|P\|$
HDS	$\max(0, 1 - 0.5\|P\|)$
EDS	$\frac{\|P\|}{e^{\|P\|}-1}$
PLDS	$\max(0, (1 - 0.5\|P\|)^5)$

Table 1: Summary of low-order numerical schemes.

3 Code Structure

The program is designed to solve the Smith-Hutton convection–diffusion transport problem by evaluating different numerical schemes and varying values of the ρ/Γ ratio. The code is organized into clearly defined blocks that allow for the simulation of the scalar field ϕ and its comparison with a reference solution.

Before running the simulation, the user can set two configuration parameters: `PlotPhi`, which enables or disables the 2D plot of $\phi(x, y)$, and `rgval`, which selects the set of ρ/Γ values to be tested. Specifically, `rgval = 1` uses the set $[10, 10^3, 10^6]$, while `rgval = 2` selects $[50, 75, 100]$. The CDS scheme (`scheme = 2`) is known to be unstable for high ρ/Γ, so when using `rgval = 1`, it is recommended to exclude that scheme to avoid runtime errors.

The main script is structured as follows:

1. **Initialization and Input Data:** Physical parameters (domain size, number of nodes, reference pressure and temperature, etc.) are defined, along with an initial value for the parameter α. The vector `rg` contains the set of ρ/Γ values to be analyzed. A reference solution (`CTTC_data`) is also loaded for later comparison.

2. **Mesh Generation and Initial Conditions:** The mesh is generated using the `SH_mesh_generator` function, and matrices such as `rho`, `x`, and `y` are initialized. The variable `phi_values` is used to store the interpolated values of ϕ at the outlet boundary ($y = 0$), while `phi_scheme` is used to save the results from all numerical schemes and ρ/Γ values.

3. **Main Loop over Schemes and ρ/Γ Values:** A loop is implemented: the outer loop iterates over the five numerical schemes (UDS, CDS, HDS, EDS, and PLDS), and the inner loop over the values of ρ/Γ. For each iteration:

   - The diffusion coefficient Γ is computed based on ρ and the current ρ/Γ value.
   - The velocity field is generated using `set_velocity_field`.
   - The discretized coefficients a_E, a_W, a_N, a_S, a_P and the source term b are computed using the `compute_coefficients` function, along with the interpolation weights $A(P)$.
   - The linear system is solved using the iterative Gauss-Seidel method (`SH_gauss_seidel`).
   - The values of ϕ at the outlet boundary are interpolated and stored for both general analysis and scheme comparison.

4. **Error Analysis:** The absolute error between the numerical results and the CTTC reference solution is calculated and stored in the `err` variable. The plot of the absolute error as a function of N=M has been done using excel, not Matlab.

5. **Results Visualization:** Finally, the results are plotted:

   - Three figures are generated, each comparing the five numerical schemes for one of the three ρ/Γ values. These plots show the variation of ϕ along the outlet boundary.
   - This visual comparison highlights how each scheme performs under different physical regimes (diffusion/ convection dominated).

4 Code Verification

The verification of the code is performed by comparing the numerical results obtained for the variable ϕ at the outlet boundary of the domain, specifically at $y = 0$ and $x \in [0, 1]$. In this region, ϕ values are evaluated for different ρ/Γ numbers, which characterize the transport regime (diffusive or convective).

Table 3 shows the reference results provided by the CTTC for $\rho/\Gamma = 10$, 10^3, and 10^6. These values have been used to validate the implementation, ensuring the correct numerical behavior of the developed solver under different flow regimes.

x-position	$\rho/\Gamma = 10$	$\rho/\Gamma = 10^3$	$\rho/\Gamma = 10^6$
0.0	1.989	2.0000	2.000
0.1	1.402	1.9990	2.000
0.2	1.146	1.9997	2.000
0.3	0.946	1.9850	1.999
0.4	0.775	1.8410	1.964
0.5	0.621	0.9510	1.000
0.6	0.480	0.1540	0.036
0.7	0.349	0.0010	0.001
0.8	0.227	0.0000	0.000
0.9	0.111	0.0000	0.000
1.0	0.000	0.0000	0.000

Table 2: Numerical results at the outlet for different values of ρ/Γ, provided by CTTC.

5 Code Results

After solving the problem, the code results are quite similar to the CTTC solution:

x-position	$\rho/\Gamma = 10$	$\rho/\Gamma = 10^3$	$\rho/\Gamma = 10^6$
0.0	1.9539	2.0000	2.0000
0.1	1.3786	2.0000	2.0000
0.2	1.1135	1.9998	2.0000
0.3	0.9074	1.9920	1.9986
0.4	0.7325	1.8266	1.9266
0.5	0.5786	0.9669	0.9784
0.6	0.4410	0.1469	0.0724
0.7	0.3168	0.0059	0.0016
0.8	0.2038	0.0001	0.0000
0.9	0.0995	0.0000	0.0000
1.0	0.0000	0.0000	0.0000

Table 3: Numerical results at the outlet for different values of ρ/Γ, provided by CTTC. N=1001, M=501

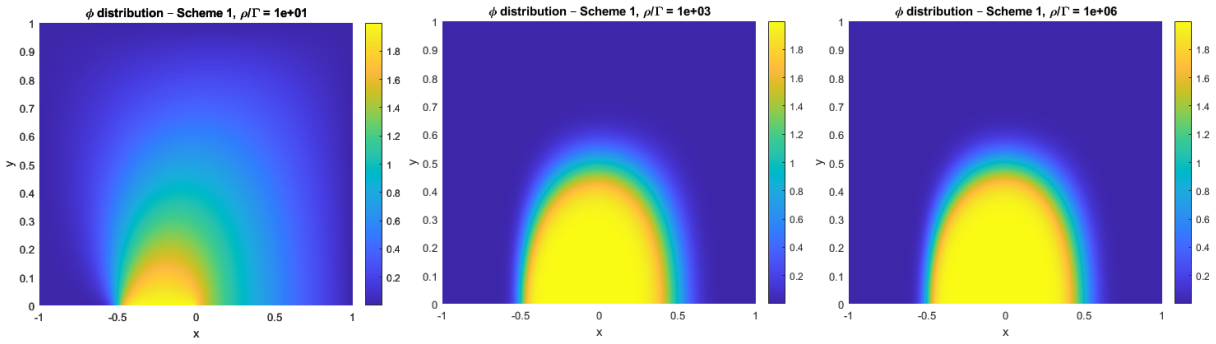


Figure 2: Phi(x,y) for $\rho/\Gamma = 10$, $\rho/\Gamma = 10^3$, $\rho/\Gamma = 10^6$. UDS scheme. N=200, M=100.

6 Numerical studies

6.1 Numerical Scheme Comparison

To analyze the influence of the discretization scheme on the results, a comparison between five different numerical schemes has been carried out: Upwind (UDS), Central (CDS), Hybrid (HDS), Exponential (EDS), and Power Law (PLDS).

For each considered value of ρ/Γ (50, 75, and 100), the profile of ϕ along the outlet boundary ($y = 0$) has been plotted as a function of the x-position. Each plot includes the results corresponding to the five schemes mentioned above in order to compare its behaviour.

As it was previously said, the CDS scheme (2) is unstable for high ρ/Γ numbers, so this study is carried out with smaller ρ/Γ values (rgval=2)

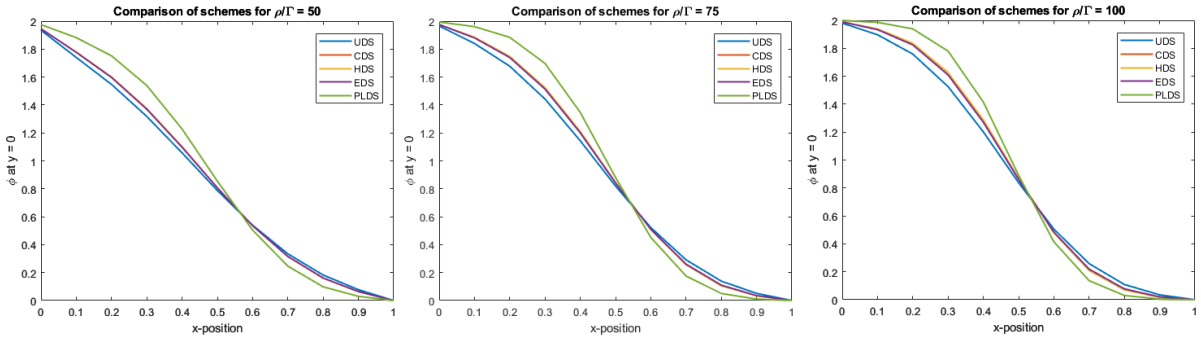


Figure 3: Comparison of five different numerical schemes (UDS, CDS, HDS, EDS, PLDS) for ρ/Γ values: 50, 75, and 100.

Scheme behavior:

- **UDS** introduces greater numerical diffusion, resulting in smoother profiles, especially in regions with steep gradients.
- **CDS**, being second-order, is more accurate in diffusion-dominated flows, but may become unstable or unreliable for high convection.
- **HDS** combines the robustness of UDS and the accuracy of CDS, providing a more balanced and less diffusive profile.
- **EDS** and **PLDS**, designed for high-Peclet number flows, better capture sharp gradients in ϕ, producing profiles closer to the expected analytical solution in convection-dominated regimes.

6.2 Mesh convergence study

In order to evaluate the convergence of the numerical solution to the Smith-Hutton problem, an error analysis has been carried out by comparing the results obtained using the HDS scheme with the reference solution provided by the CTTC. For this purpose, the mean absolute error between both solutions has been computed for several uniform mesh refinements (with $N = M$) and for different values of the dimensionless number ρ/Γ: 10, 10^3, and 10^6.

- The error decreases as the mesh is refined, i.e., as the number of nodes increases. This confirms that the implemented numerical method is convergent.
- This trend is consistent for all values of ρ/Γ, although with varying magnitude.
- For $\rho/\Gamma = 10$, corresponding to a diffusion-dominated regime, the error is significantly lower even with coarse meshes. This suggests that diffusion smooths the transported variable, making the numerical approximation easier.

- For high values of ρ/Γ (such as 10^3 or 10^6), where convection dominates, the initial error is larger and the reduction of error with refinement is slower, indicating the greater numerical challenge of convection-dominated regimes.
- Even in the most demanding cases, the error consistently decreases as the mesh resolution increases, validating the stability of the applied method.

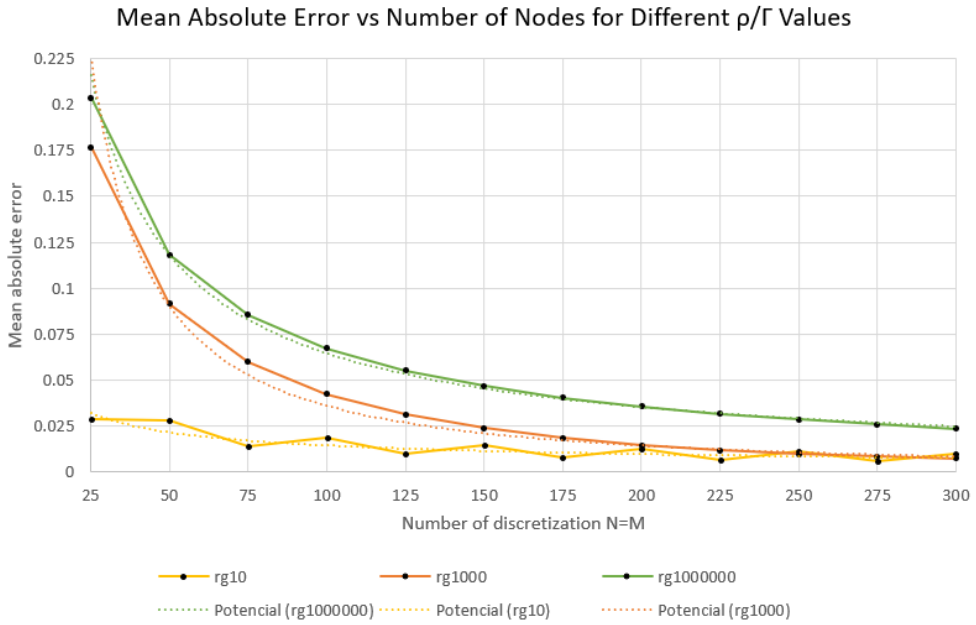


Figure 4: Mean absolute error of the numerical solution versus the number of discretization nodes (N = M) for different ρ/Γ values using the Hybrid Differencing Scheme (HDS).

7 Annexes

```
1
2  % ANTONIO LUQUE L PEZ
3  % MUEA
4  % ASSIGNMENT 2: SMITH-HUTTON RESOLUTION
5  %
       ----------------------------------------------------------------------
6
7  clear workspace,
8  clear all;
9  clc;
10 clearvars;
11 close all;
12 %
       ----------------------------------------------------------------------
13
14 %%INSTRUCTIONS%%
15
16 %To plot Phi(x,y)-->plotPhi=1. If not, plotPhi=0(Recommended)
17 %For rg=[50, 75, 100]-->rgval=2. For rg=[10, 1e3, 1e6]-->rgval=1.(CDS
       scheme is not valid for high   /  -->Code crashes in line 68 becouse
       the "for" visits the scheme 2(CDS). To use rgval=1, change the "1:5"
       for a value different to 2.)
18
19 PlotPhi=0; rgval=2;
20
21 %
       ----------------------------------------------------------------------
22
23 % 1. INPUT DATA
24 % 1.1 Physical data
25 L=2;
26 H=1;
27 alpha=10;
28
29 % 1.2 Numerical data
30 N = 100;
31 M = 50;
32 delta=1e-10;
33 scheme=1;
34
35 % 2. PREVIOUS CALCULATIONS
36 dx = L/N;
37 dy = H/M;
```

```
38
39 [x, y] = SH_mesh_generator(N,M,L,H); % Function that creates the
       geometry of the mesh.
40 [X, Y] = meshgrid(x, y);
41
42 R=287;
43 P_ref=101325;
44 T_ref=300;
45
46 rho=ones(M+2,N+2)*(P_ref/(R*T_ref));
47
48 if rgval==1
49 rg = [10, 1e3, 1e6]; %   /   values 1
50 else
51 rg = [50, 75, 100]; %   /   values 2
52 end
53
54 CTTC_data = [ % values given as a solution for Phi(  /  =[10, 1e3, 1e6])
55     1.9890, 2.0000, 2.0000;
56     1.4020, 1.9990, 2.0000;
57     1.1460, 1.9997, 2.0000;
58     0.9460, 1.9850, 1.9990;
59     0.7750, 1.8410, 1.9640;
60     0.6210, 0.9510, 1.0000;
61     0.4800, 0.1540, 0.0360;
62     0.3490, 0.0010, 0.0010;
63     0.2270, 0.0000, 0.0000;
64     0.1110, 0.0000, 0.0000;
65     0.0000, 0.0000, 0.0000];
66
67 x_output = 0:0.1:1;
68 phi_values = zeros(length(x_output), length(rg));  % almacenar
       interpolaciones
69 phi_scheme=zeros(length(x_output),5*length(rg));
70
71 % 3. COMPUTE PHI MATRIX
72 for scheme=1:5 %Loop for plotting the differents phi values depending on
       the numerical scheme for each rho/Gamma values
73     for k = 1:length(rg)%Loop for creating the Phi(x) table depending on
           rho/Gamma values
74         rgi = rg(k);
75         Gamma = (rho / rgi) .* ones(M+2, N+2);
76
77         [u,v] = set_velocity_field(N, M, x, y); % Creates the velocity
               field
78         [a_E, a_W, a_N, a_S, a_P, b, AP_e, AP_w, AP_n, AP_s] =
               compute_coefficients(u, v, rho, Gamma, x, y, dx, dy, N, M,
               scheme); % Compute the discretization coefficients
```

```
79          [Phi, iter, error] = SH_gauss_seidel(a_E, a_W, a_N, a_S, a_P, b,
                delta, M, N, alpha, x); % Solve the discretized equation
80
81          % Creates the Phi interpolation for x=[0, 0.1, 0.2, ..., 0.9, 1]
82          a = ceil((N+2)/2+1) : N+2;
83          x_cols = x(a);
84          Phi_output = Phi(1, a);
85          Phi_interp = interp1(x_cols, Phi_output, x_output, 'makima');
86
87          phi_values(:, k) = Phi_interp;
88
89              %phi_scheme creates a matrix with Phi for each scheme for
                    each rho/gamma
90              c = (scheme-1)*length(rg) + k;
91              phi_scheme(:, c) = Phi_interp;
92
93          %Plot Phi(x,y)
94          if PlotPhi==1
95          figure;
96          pcolor(X, Y, Phi);
97          shading interp;
98          colorbar;
99          title(sprintf('\\phi distribution    Scheme %d, \\rho/\\Gamma =
                %.0e', scheme, rgi));
100         xlabel('x'); ylabel('y');
101         end
102     end
103 end
104
105 table_rg(x_output, phi_values, rg); %Function that prints the table with
        Phi for each rho/gamma
106
107
108 err=abs(phi_values-CTTC_data); %Compute the absolute error of Phi
109
110 %Plot a graphic for comparing eache scheme for each rho/gamma
111 if rgval==2
112 figure;
113 plot(x_output, phi_scheme(:, [1, 4, 7, 10, 13]), 'LineWidth', 1.5)
114 title('Comparison of schemes for \rho/\Gamma = 50')
115 xlabel('x-position')
116 ylabel('\phi at y = 0')
117 legend('UDS', 'CDS', 'HDS', 'EDS', 'PLDS', 'Location', 'best');
118
119 figure;
120 plot(x_output, phi_scheme(:, [2, 5, 8, 11, 14]), 'LineWidth', 1.5)
121 title('Comparison of schemes for \rho/\Gamma = 75')
122 xlabel('x-position')
```

```
ylabel('\phi at y = 0')
legend('UDS', 'CDS', 'HDS', 'EDS', 'PLDS', 'Location', 'best');

figure;
plot(x_output, phi_scheme(:, [3, 6, 9, 12, 15]), 'LineWidth', 1.5)
title('Comparison of schemes for \rho/\Gamma = 100')
xlabel('x-position')
ylabel('\phi at y = 0')
legend('UDS', 'CDS', 'HDS', 'EDS', 'PLDS', 'Location', 'best');

end
```

```

function [a_E, a_W, a_N, a_S, a_P, b, AP_e, AP_w, AP_n, AP_s] =
    compute_coefficients(u, v, rho, Gamma, x, y, dx, dy, N, M, scheme)

% DESCRIPTION:
%   This function computes the coefficients of the discretized
%   convection-diffusion. It supports multiple numerical schemes
%   (UDS, CDS, HDS, EDS, PLDS) to approximate the convective fluxes
%   based on the local Peclet number.
%
%   The function returns the coefficients of each cell face (a_E, a_W,
%   a_N, a_S), the central coefficient (a_P), the right-hand side term b
    ,
%   and the values A(|P|) used in the numerical schemes.
%
% INPUTS:
%   u, v       - Velocity components in x and y directions (2D matrices)
%   rho        - Density field (assumed constant or given as 2D matrix)
%   Gamma      - Diffusivity field (2D matrix)
%   x, y       - Node coordinates in x and y directions
%   dx, dy     - Grid spacing in x and y directions
%   N, M       - Number of cells in x and y
%   scheme     - Integer indicating the numerical scheme:
%                  1 - UDS (Upwind)
%                  2 - CDS (Central)
%                  3 - HDS (Hybrid)
%                  4 - EDS (Exponential)
%                  5 - PLDS (Power-law)
%
% OUTPUTS:
%   a_E, a_W, a_N, a_S, a_P, b - Discretization coefficients
%   AP_e, AP_w, AP_n, AP_s - A(|P|) values for each face
%-------------------------------------------------------------

% Matrix preallocation
F_e = zeros(M+2, N+2);
```

```
35 F_w = zeros(M+2, N+2);
36 F_n = zeros(M+2, N+2);
37 F_s = zeros(M+2, N+2);
38
39 D_e = zeros(M+2, N+2);
40 D_w = zeros(M+2, N+2);
41 D_n = zeros(M+2, N+2);
42 D_s = zeros(M+2, N+2);
43
44 a_E = zeros(M+2, N+2);
45 a_W = zeros(M+2, N+2);
46 a_N = zeros(M+2, N+2);
47 a_S = zeros(M+2, N+2);
48 a_P = zeros(M+2, N+2);
49 b   = zeros(M+2, N+2);
50
51 for i = 2:N+1
52     for j = 2:M+1
53
54         % Flujos convectivos
55         F_e(j,i) = 0.5 * (rho(j,i+1)*u(j,i+1) + rho(j,i)*u(j,i)) * dy;
56         F_w(j,i) = 0.5 * (rho(j,i)*u(j,i) + rho(j,i-1)*u(j,i-1)) * dy;
57         F_n(j,i) = 0.5 * (rho(j+1,i)*v(j+1,i) + rho(j,i)*v(j,i)) * dx;
58         F_s(j,i) = 0.5 * (rho(j,i)*v(j,i) + rho(j-1,i)*v(j-1,i)) * dx;
59
60         % Coeficientes difusivos
61         D_e(j,i) = (0.5 * (Gamma(j,i+1) + Gamma(j,i)) * dy) / (x(i+1) -
               x(i));
62         D_w(j,i) = (0.5 * (Gamma(j,i) + Gamma(j,i-1)) * dy) / (x(i) - x(
               i-1));
63         D_n(j,i) = (0.5 * (Gamma(j+1,i) + Gamma(j,i)) * dx) / (y(j+1) -
               y(j));
64         D_s(j,i) = (0.5 * (Gamma(j,i) + Gamma(j-1,i)) * dx) / (y(j) - y(
               j-1));
65
66         % Peclet number for each face
67         P_e = F_e(j,i) / D_e(j,i);
68         P_w = F_w(j,i) / D_w(j,i);
69         P_n = F_n(j,i) / D_n(j,i);
70         P_s = F_s(j,i) / D_s(j,i);
71
72         % A(|P|) according to the numerical scheme
73         switch scheme
74             case 1  % UDS
75                 AP_e = 1;
76                 AP_w = 1;
77                 AP_n = 1;
78                 AP_s = 1;
```

```
79
80              case 2  % CDS
81                  AP_e = 1 - 0.5 * abs(P_e);
82                  AP_w = 1 - 0.5 * abs(P_w);
83                  AP_n = 1 - 0.5 * abs(P_n);
84                  AP_s = 1 - 0.5 * abs(P_s);
85
86              case 3  % HDS
87                  AP_e = max(0, 1 - 0.5 * abs(P_e));
88                  AP_w = max(0, 1 - 0.5 * abs(P_w));
89                  AP_n = max(0, 1 - 0.5 * abs(P_n));
90                  AP_s = max(0, 1 - 0.5 * abs(P_s));
91
92              case 4  % EDS
93                  AP_e = abs(P_e) / (exp(abs(P_e)) - 1 + 1e-10);
94                  AP_w = abs(P_w) / (exp(abs(P_w)) - 1 + 1e-10);
95                  AP_n = abs(P_n) / (exp(abs(P_n)) - 1 + 1e-10);
96                  AP_s = abs(P_s) / (exp(abs(P_s)) - 1 + 1e-10);
97
98              case 5  % PLDS
99                  AP_e = max(0, (1 - 0.5 * abs(P_e))^5);
100                 AP_w = max(0, (1 - 0.5 * abs(P_w))^5);
101                 AP_n = max(0, (1 - 0.5 * abs(P_n))^5);
102                 AP_s = max(0, (1 - 0.5 * abs(P_s))^5);
103
104             otherwise
105                 error('Esquema num rico no reconocido');
106         end
107
108         % Compute a_i coefficients
109         a_E(j,i) = D_e(j,i) * AP_e + max(-F_e(j,i), 0);
110         a_W(j,i) = D_w(j,i) * AP_w + max( F_w(j,i), 0);
111         a_N(j,i) = D_n(j,i) * AP_n + max(-F_n(j,i), 0);
112         a_S(j,i) = D_s(j,i) * AP_s + max( F_s(j,i), 0);
113
114         a_P(j,i) = a_E(j,i) + a_W(j,i) + a_N(j,i) + a_S(j,i);% Steady-
              state without source term
115         b(j,i)   = 0; % Steady-state without source term
116
117     end
118 end
119 end
```

```
1
2 function [u_P,v_P] = set_velocity_field(N, M, x_P, y_P)
3
4 % Computes the analytical velocity field for the Smith-Hutton problem
5 % at the cell centers of a 2D structured mesh.
```

```
%
%   u(x,y) =  2y(1 - x  )
%   v(x,y) = -2x(1 - y  )
%
% Inputs:
%   - N, M   : Number of divisions in x and y directions
%   - x_P, y_P : Coordinates of cell centers in x and y directions
%
% Outputs:
%   - u_P, v_P : 2D velocity field components at cell centers

u_P=zeros(M+2,N+2);
v_P=zeros(M+2,N+2);

for i=1:N+2
    for j=1:M+2
u_P(j,i)=2*y_P(j)*(1-x_P(i)^2);
v_P(j,i)=-2*x_P(i)*(1-y_P(j)^2);
    end
end

end
```

```

function [Phi, iter, error] = SH_gauss_seidel(a_E, a_W, a_N, a_S, a_P, b
    , delta, M, N, alpha, x)

% Solves the 2D convection-diffusion equation using the
% Gauss-Seidel iterative method for a structured mesh.
%
% The algorithm updates the scalar field Phi at the internal nodes using
% the discrete equation coefficients and applies Dirichlet and Neumann
% boundary conditions at the inlet and outlet, respectively.
%
% Inputs:
%   - a_E, a_W, a_N, a_S : Discrete coefficients for each face
%   - a_P                : Central coefficient for each node
%   - b                  : Right-hand side (source term)
%   - delta              : Convergence tolerance
%   - M, N               : Number of cells in y and x directions
%   - alpha              : Parameter controlling the inlet boundary
    profile
%   - x                  : x-coordinate of cell centers
%
% Outputs:
%   - Phi   : Scalar field solution (2D matrix)
%   - iter  : Number of iterations performed
%   - error : Final error (residual) of the iterative method
```

```
%-----------------------------------------------------------

Phi=ones(M+2, N+2)*(1 - tanh(alpha));

max_iter = 100000;
error = Inf;

for iter = 1:max_iter
    phi_old = Phi;  % Guardamos el valor anterior para evaluar el error


        %Inlet
        for i=1:ceil((N+2)/2)
        Phi(1,i)= 1 + tanh (alpha*(2*x(i) + 1));
        end

    for j = 2:M+1
        for i = 2:N+1
            Phi(j, i) = (a_E(j, i) * Phi(j, i+1) + ...
                         a_W(j, i) * Phi(j, i-1) + ...
                         a_N(j, i) * Phi(j+1, i) + ...
                         a_S(j, i) * Phi(j-1, i) + ...
                         b(j, i)) / a_P(j, i);
        end
    end

        %Outlet
        for i=ceil((N+2)/2+1):N+2
        Phi(1,i)= Phi(2,i);
        end

    % Convergence criteria
    error = max(max(abs(Phi - phi_old)));

    if error < delta
        break;
    end
end

end
```

```

function [x_P,y_P] = SH_mesh_generator(N, M, L, H)

% Generates the coordinates of cell centers (P nodes) in a 2D structured
    mesh
% over a domain of size L    H, with N divisions along x and M along y.
% Includes ghost cells at the boundaries.
```

```
%
% Inputs:
%   - N, M : Number of divisions in x and y directions
%   - L, H : Physical dimensions of the domain
%
% Outputs:
%   - x_P, y_P : Vectors with the coordinates of the cell centers
%-------------------------------------------------------

x_P=zeros(1, N+2);
x_P(2:end-1) = (0.5-N/2:1:N/2-0.5) * (L/N);
x_P(1) = -L/2;
x_P(end) = L/2;

y_P=zeros(1, M+2);
y_P(2:end-1) = (0.5:1:M-0.5) * (H/M);
y_P(1) = 0;
y_P(end) = H;

end
```

```

function table_rg(x_output, phi_values, rg)

% Displays a table in the console with the values of the scalar field Phi
% at the outlet for different x positions and for different values of the ratio   /
%
% Inputs:
%   - x_output   : Vector with x positions at the outlet
%   - phi_values : Matrix with Phi values
%                  (each column corresponds to a different   /  )
%   - rg         : Vector with the tested   /   values
%-------------------------------------------------------

    % Encabezado
    fprintf('\n%-10s', 'x-position');
    for j = 1:length(rg)
        fprintf('%16s', [' /  ␣=␣', num2str(rg(j), '%.0e')]);
    end
    fprintf('\n');

    % Filas
    for i = 1:length(x_output)
        fprintf('%-10.1f', x_output(i));
        for j = 1:length(rg)
            fprintf('%16.4f', phi_values(i, j));
```

```
26          end
27          fprintf('\n');
28      end
29  end
```